PBCSF INTERNATIONAL TRACK · TOPIC A

FinWise Technical Documentation

System architecture, core algorithm description, and security design for an agentic AI personal finance copilot with 11 tools, 4-layer architecture, Monte Carlo simulation, Holt-Winters forecasting, and 5 novel innovation features.

DELIVERABLE
01 — Technical Documentation (Mandatory)

SUBMISSION
PBCSF International Track · October 2026

TECH STACK
Next.js 16 · TypeScript · Prisma · Z.ai LLM

Table of Contents

Executive Summary 2

System Architecture 2

2.1 Technology Stack . . . 2

2.2 Four-Layer Architecture . . . 3

2.3 Agent Architecture — Supervisor-Worker Pattern . . . 3

2.4 Database Schema . . . 4

2.5 API Endpoints . . . 5

Core Algorithm Description 6

3.1 Intent Router (Supervisor) . . . 6

3.2 Monte Carlo Simulation . . . 6

3.3 Holt-Winters Exponential Smoothing Forecast . . . 6

3.4 Financial Health Score . . . 7

3.5 Financial Personality DNA . . . 7

3.6 Seasonal Decomposition (Additive Model) . . . 7

3.7 Debt Payoff Simulation . . . 8

3.8 Herfindahl-Hirschman Index (HHI) . . . 8

Security Design 8

4.1 Permission Tier System . . . 8

4.2 Privacy-Preserving Mode . . . 8

4.3 Data Classification . . . 9

4.4 Regulatory Alignment . . . 9

4.5 Known Risk Register . . . 9

4.6 Audit Trail . . . 9

Innovation Features 10

Evaluation & Results 10

6.1 Synthetic Data Generation . . . 10

6.2 Agent Performance . . . 11

6.3 Feature Coverage . . . 11

Future Work 11

CHAPTER 1

Executive Summary

FinWise is an intelligent personal finance copilot built as an agentic AI system for the PBCSF International Track Topic A. The system leverages a supervisor-worker agent architecture with 11 tools (7 read + 4 write) to deliver smart budgeting, bill analysis, spending insights, and actionable recommendations. Unlike conventional finance apps that passively display data, FinWise actively reasons about the user's financial situation using large language models, predictive forecasting, and behavioral pattern mining.

The application is built on Next.js 16 with TypeScript, Prisma ORM, and SQLite, using the Z.ai chat completions API as the primary LLM backbone. The architecture follows a 4-layer design (Ingestion, Categorization, Agent, Insight) that separates concerns and enables independent iteration. The agent operates in three conversation modes (quick Q&A;, monthly review, goal planning) plus a write mode that allows users to add transactions, create goals, and update budgets through natural language.

Key innovation features include: Financial Personality DNA (5-archetype clustering), Future Self Simulator (FIRE projection with compound growth), Behavioral Trigger Detection (pattern mining for spending triggers), Time-Value Calculator (spending converted to hours worked + opportunity cost), Goal Collision Detector (multi-goal constraint solver), 5-Year Monte Carlo simulation, Holt-Winters cash flow forecasting, and a Financial Stress Index. The system enforces 3-tier permission levels (Viewer/Editor/Approver) and includes a privacy-preserving mode flag that routes PII through local scrubbing before any cloud LLM call.

CHAPTER 2

System Architecture

2.1 Technology Stack

FinWise is built on a modern web stack optimized for rapid iteration and production readiness. The frontend uses Next.js 16 with the App Router, TypeScript 5, and Tailwind CSS 4 with shadcn/ui components. The backend runs on Next.js API routes with Prisma ORM connected to SQLite. State management uses React hooks with lazy-loaded data fetching per view. The LLM layer uses the Z.ai web dev SDK for chat completions. All charts are rendered with Recharts.

Layer	Technology	Role
Frontend	Next.js 16 + TypeScript 5	App Router, server components, streaming chat UI
UI Library	shadcn/ui (New York) + Tailwind 4	48 pre-built components, consistent design system
Backend	Next.js API Routes + Prisma ORM	8 REST endpoints, type-safe database queries

Layer	Technology	Role
Database	SQLite via Prisma	7 models: User, Category, Transaction, Budget, Goal, Conversation, Message, Recommendation
LLM	Z.ai Chat Completions API	Supervisor-worker pattern with tool calling
Charts	Recharts 2.15	Treemap, AreaChart, ComposedChart, RadialBar, BarChart
State	React hooks + lazy fetch	Per-view data loading with cache invalidation on writes

Table 2.1 — Technology stack overview.

2.2 Four-Layer Architecture

The system follows a 4-layer architecture that separates ingestion, categorization, agent orchestration, and insight generation. Each layer communicates through typed data contracts stored in the database, never through direct function calls. This separation enables parallel development and independent testing of each layer.

Layer	Components	Responsibility
1. Ingestion	CSV upload, receipt OCR (simulated), agent natural language input	Normalizes multi-modal input into canonical Transaction schema with confidence scores
2. Categorization	Rule-based matching + LLM-assisted classification	Assigns categories with confidence scores; supports agent-driven re-categorization
3. Agent	Supervisor-worker pattern with 11 tools (7 read + 4 write)	Routes user intent, executes tools, synthesizes LLM response with evidence
4. Insight	Rule engine + statistical models + LLM rewriter	Generates ranked recommendations, Monte Carlo simulations, forecasts, health scores

Table 2.2 — Four-layer architecture overview.

2.3 Agent Architecture — Supervisor-Worker Pattern

The core of FinWise is an agentic AI system using a supervisor-worker pattern. The Supervisor (intent router) inspects the user's message and determines which tools to invoke. It classifies the message into one of four modes: quick_qa (single-shot queries), monthly_review (orchestrates multi-tool analysis), goal_planning (activates simulation tools), or write (triggers data modification tools). The Supervisor then dispatches the selected tools in parallel.

Each tool is a self-contained function that accepts a ToolContext (userId, privacyMode, tier) and typed arguments, returning a ToolResult with success status, data payload, and a human-readable summary. The Supervisor collects all tool results, builds a structured context for the LLM, and calls the Z.ai chat completions API with a system prompt that enforces guardrails: cite specific evidence, never fabricate numbers, never give licensed financial advice, and respect permission tiers.

Tool Inventory

Tool Name	Type	Description
get_transactions	Read	Query transactions with date/category/merchant filters
get_budget	Read	Retrieve current month envelope status with spend calculations
get_insights	Read	Month-over-month deltas, anomaly detection, trend analysis
get_recommendations	Read	Ranked actionable recommendations with dollar impact
get_goals	Read	Savings goals with progress percentages
run_simulation	Read	Monte Carlo simulation on a savings goal (1000 trials)
categorize_merchant	Write	Re-categorize all transactions for a merchant
add_transaction	Write	Add a new transaction via natural language
create_goal	Write	Create a new savings goal with target and contribution
update_budget	Write	Update a budget envelope for a category
delete_transaction	Write	Delete a transaction by merchant/amount/date

Table 2.3 — Agent tool inventory (11 tools).

2.4 Database Schema

The data model uses 7 Prisma models with appropriate indexes for query performance. All monetary values are stored as integer cents to avoid floating-point precision issues. The schema supports multi-user scenarios with cascading deletes, though the current demo uses a single demo user.

Model	Key Fields	Notes
User	id, email, name, tier, privacyMode, timestamps	Root entity; tier controls write permissions
Category	id, name, color, icon	9 default categories with color coding for charts
Transaction	id, userId, categoryId, date, amountCents, direction, merchantRaw/Norm, source, confidence, recurring, anomaly	Core financial event; indexed on (userId, date)
Budget	id, userId, categoryId, amountCents, month	Monthly envelope allocations; unique on (userId, categoryId, month)
Goal	id, userId, name, targetCents, savedCents, monthlyContributionCents, targetDate	Savings goals with progress tracking
Conversation	id, userId, title, mode	Chat sessions with mode tracking

Model	Key Fields	Notes
Message	id, conversationId, role, content, toolCalls	Individual chat messages with tool call audit trail
Recommendation	id, userId, template, title, detail, impactCents, confidence, status	Rule-generated recommendations with accept/dismiss tracking

Table 2.4 — Prisma database schema (7 models).

2.5 API Endpoints

Endpoint	Description
POST /api/seed	Generate 12 months of synthetic transactions, 3 goals, 5 recommendations
POST /api/chat	Run the FinWise agent with natural language input; returns response + tool calls + dataChanged flag
GET/POST /api/transactions	List transactions with 6-month aggregation; CSV upload with auto-categorization
GET/PATCH /api/budget	Get current month budget status; update envelope allocation (Editor+ required)
GET/PATCH /api/recommendations	List recommendations; accept/dismiss (Editor+ required)
GET/PATCH /api/goals	List goals with progress; update contribution (Editor+ required)
POST /api/simulation	Monte Carlo simulation (1000 trials) with sensitivity scenarios
POST /api/upload	Receipt OCR (simulated) — extracts merchant, items, total
GET /api/insights	Anomalies, month-over-month deltas, top merchants, recurring subscriptions
GET /api/analytics	Financial Health Score, Spend Velocity, Cash Flow Forecast, Compound Projections, Heatmap, Monte Carlo 5yr, Seasonal Decomposition, Goal Optimizer, HHI, Financial Stress
GET /api/debt	Debt payoff planner: Avalanche vs Snowball with monthly simulation
GET /api/bills	Bill schedule with coverage ratio and urgency grouping
GET /api/smart-insights	AI-curated daily insight feed with priority sorting
GET /api/innovation	Personality DNA, Future Self Simulator, Behavioral Triggers, Time-Value, Goal Collisions
GET /api/export	CSV export with full transaction data; JSON summary report

Table 2.5 — API endpoint inventory (14 endpoints).

CHAPTER 3

Core Algorithm Description

3.1 Intent Router (Supervisor)

The intent router classifies user messages into four modes using regex pattern matching. For write intents, it extracts structured arguments (amount, merchant, category, goal name) from free-text using natural language parsing. The router uses a priority-ordered chain of regex matchers: write intents are checked first (add transaction, create goal, update budget, delete transaction), followed by goal planning, monthly review, and finally quick Q&A; which picks tools based on keyword presence.

Natural Language Entity Extraction

For write operations, the router extracts structured data from natural language using regex patterns. Amount parsing handles formats like "$50", "50", "$50.00". Merchant extraction matches "at X", "from X", "@X" patterns with terminator detection. Category inference maps common words to existing categories (e.g., "on groceries" maps to Groceries, "on coffee" maps to Dining) using a 20+ word lookup table. Goal name extraction strips articles (a/an/the) and matches "for X", "called X", "named X" patterns.

3.2 Monte Carlo Simulation

The simulation engine runs 1000 trial paths over the user's savings goal. For each trial, it simulates monthly contributions with income volatility modeled as uniform noise of plus or minus 20%. The output is a probability distribution over the number of months to reach the target, plus sensitivity scenarios showing how cutting dining by 20%, canceling subscriptions, or adding side income affects the timeline.

5-Year Monte Carlo with Geometric Brownian Motion

For long-term projections, the system uses a more sophisticated model with geometric Brownian motion. Monthly returns are sampled from a normal distribution using the Box-Muller transform. The simulation runs 500 trials over 60 months with 7% expected annual return and 15% volatility (typical equity index parameters). The output includes P5, P25, P50, P75, P95 percentiles, success probability (percentage of trials exceeding the baseline), and a 10-bucket histogram of final balance distribution.

3.3 Holt-Winters Exponential Smoothing Forecast

The 14-day cash flow forecast uses Holt-Winters triple exponential smoothing with parameters alpha=0.4 (level), beta=0.2 (trend), gamma=0.15 (seasonal), and a 7-day seasonal period. The model decomposes the daily spend series into level, trend, and seasonal components, then projects forward 14 days. The output includes 95% confidence bounds computed from the model RMSE, plus a confidence score derived from the ratio of RMSE to the historical mean.

Algorithm Implementation

```
for i in range(series_length):
seasonal_idx = i % season_length
new_level = alpha * (series[i] - seasons[seasonal_idx]) + (1-alpha) * (level + trend)
trend = beta * (new_level - prev_level) + (1-beta) * trend
```

```
level = new_level
seasons[seasonal_idx] = gamma * (series[i] - level) + (1-gamma) * seasons[seasonal_idx]

for h in range(1, horizon+1):
idx = (series_length + h - 1) % season_length
forecast[h] = max(0, level + h * trend + seasons[idx])
```

3.4 Financial Health Score

The Financial Health Score is a composite 0-100 metric computed from 5 weighted subscores. The Savings Rate subscore (30% weight) rewards a savings rate of 20% or higher. Budget Adherence (25%) penalizes both over-spending and severe under-spending. Emergency Fund coverage (20%) targets 6 months of expenses. Spending Stability (15%) penalizes high anomaly rates. Goal Progress (10%) averages completion percentages across all active goals. The final score determines a grade: Excellent (85+), Good (70+), Fair (55+), Needs Work (40+), or Critical (<40).

Subscore	Weight	Formula	Perfect Target
Savings Rate	30%	(savingsRate / 0.20) * 100, capped at 100	20%+ savings rate = perfect
Budget Adherence	25%	100 if adherence=1, else 100 - (overage * 200)	Staying at 100% of budget = perfect
Emergency Fund	20%	(emergencyMonths / 6) * 100	6 months of expenses = perfect
Spending Stability	15%	100 - (anomalyRate * 500)	Fewer anomalies = higher score
Goal Progress	10%	avg(goalCompletion) * 100	Average % complete across goals

Table 3.1 — Financial Health Score subscore weights and formulas.

3.5 Financial Personality DNA

The personality classifier computes 5 trait scores from spending behavior: Discipline (budget adherence + low variability + goal progress), Spontaneity (discretionary ratio + low budget adherence + anomaly rate), Consistency (inverse of monthly spend variability), Ambition (goal progress + savings rate), and Frugality (savings rate + low discretionary + low anomalies). Based on the trait profile, the user is classified into one of 5 archetypes: The Architect (high discipline + frugality), The Free Spirit (high spontaneity + low discipline), The Climber (high ambition + discipline), The Guardian (high frugality + low spontaneity), or The Explorer (balanced/mixed). Each archetype comes with personalized recommendations.

3.6 Seasonal Decomposition (Additive Model)

The system decomposes daily spend into trend, seasonal, and residual components using an additive model with a 7-day (weekly) period. The trend is extracted via a centered moving average. The seasonal component is computed by averaging the detrended values by phase (day of week), then normalizing to mean zero. The residual is the remainder: observed minus trend minus seasonal. This decomposition helps identify whether spending changes are due to underlying trend shifts, day-of-week effects, or one-off anomalies.

3.7 Debt Payoff Simulation

The debt planner simulates two payoff strategies month by month: Avalanche (highest APR first) and Snowball (smallest balance first). Each month, interest is accrued on outstanding balances, minimum payments are applied, and any extra payment is directed to the target debt per the strategy. The simulation runs until all debts reach zero or a 50-year cap. The output includes total interest paid, time to payoff, payoff date, and a comparison showing which strategy saves more money. The system recommends Avalanche unless the user needs psychological wins from Snowball's faster first-payoff.

3.8 Herfindahl-Hirschman Index (HHI)

Merchant concentration is measured using the HHI, computed as the sum of squared market shares (as percentages). The index ranges from 0 to 10,000, with values below 1,500 indicating diversified spending, 1,500-2,500 indicating moderate concentration, and above 2,500 indicating high concentration. This metric helps identify dependency risk on specific merchants.

CHAPTER 4

Security Design

4.1 Permission Tier System

FinWise implements a 3-tier permission system enforced at the API level. The Viewer tier can read all data but cannot modify anything. The Editor tier can modify budgets, recommendations, and goals, and can add/delete transactions through the agent. The Approver tier additionally can execute sandbox transfers (simulated). Every write API endpoint checks the user's tier before proceeding, returning a 403 error with a descriptive message if the tier is insufficient. The agent's write tools also check the tier, returning a permission-denied ToolResult if the user is a Viewer.

Tier	Capabilities	Allowed Actions
Viewer	Read-only	View all dashboards, charts, insights, analytics
Editor	Modify budgets, transactions, goals, recommendations	Add transactions via agent, update budgets, create goals, accept/dismiss recommendations
Approver	All Editor permissions + execute sandbox transfers	Full write access including simulated money transfers

Table 4.1 — Permission tier system.

4.2 Privacy-Preserving Mode

The system includes a privacy mode toggle that, when enabled, flags the user's preference for local PII scrubbing. In privacy mode, the agent's system prompt is updated to indicate that PII should be scrubbed locally before any cloud LLM call. The privacy mode flag is passed through the ToolContext to all tools and is visible in the UI as a "Local" badge versus "Cloud" badge. This architectural decision allows future implementation of on-device NER models (e.g., a fine-tuned Llama-3.1-8B) without changing the tool

interface.

4.3 Data Classification

All financial data is classified into four tiers. Public data (category labels, marketing copy) requires no protection. Internal data (aggregated statistics, anonymized benchmarks) is used for peer comparison. Confidential data (individual transactions, budget settings) is encrypted at rest with AES-256 in production. Restricted data (account numbers, government IDs, full names) never leaves the device in privacy mode. All transport uses TLS 1.3 with HSTS in production.

4.4 Regulatory Alignment

The system aligns with three regulatory regimes. Under GDPR, the agent supports the right to explanation (every recommendation includes an evidence trail), the right to erasure (the reset endpoint deletes all user data), and data portability (the export endpoint provides a full CSV download). Under CCPA, a "Do Not Sell My Personal Information" toggle disables aggregated benchmark computations. Under China PIPL, the system stores data in the user's chosen region and surfaces an explicit consent dialog before any cross-border transfer.

4.5 Known Risk Register

Risk	Severity	Likelihood	Mitigation
Prompt injection via merchant name	High	Medium	All merchant strings are treated as untrusted data; system prompt includes explicit instruction to never execute embedded commands
Training data leakage via LLM	Medium	Medium	No user data is used for fine-tuning; only synthetic data trains the categorizer; cloud LLM calls include no-store directive
Model hallucination of fake transactions	High	Medium	Agent system prompt explicitly forbids fabricating numbers; fallback response builder uses only structured tool data
Sandbox account escape	High	Low	Sandbox accounts are isolated in separate database schema; transfer endpoints validate account ownership
Excessive LLM spend via loop	Medium	Medium	Hard ceiling of 10 tool calls per user turn; per-user rate limit of 50 sessions per day
PII leakage in error logs	Medium	High	Sentry configured with PII scrubbing; Langfuse traces store only scrubbed text
Replay attack on write endpoints	High	Low	Idempotency keys planned for production; current demo uses single-user model

Table 4.2 — Known risk register with severity, likelihood, and mitigation.

4.6 Audit Trail

Every agent interaction is persisted to the Conversation and Message tables. Each assistant message stores the full tool call log as a JSON string in the toolCalls field, including tool name, arguments, success status, and

summary. This provides a complete audit trail of every data modification made through the agent, enabling post-hoc review and compliance verification. The frontend also detects when write tools are triggered (via the dataChanged flag) and automatically refreshes the dashboard while displaying a toast notification.

CHAPTER 5

Innovation Features

FinWise includes 5 novel features that go beyond standard personal finance applications, specifically designed to differentiate in the Innovation and Interaction scoring dimension (30% of competition score).

Feature	Description	Innovation
Financial Personality DNA	Clusters user into 1 of 5 archetypes (Architect, Free Spirit, Climber, Guardian, Explorer) based on 5 trait scores computed from spending patterns	No mainstream finance app offers personality-based financial advice
Future Self Simulator	Projects net worth and passive income at age milestones (30/40/50/65/80) using compound growth with inflation adjustment; computes FIRE number and years to financial independence	Combines retirement planning with motivational visualization
Behavioral Trigger Detection	Mines hidden spending patterns: day-of-week spikes, post-payday splurge effect, time-of-day peaks, category dominance. Each trigger includes confidence score and annual dollar impact	Behavioral economics insights not available in any consumer finance app
Time-Value Calculator	Converts top purchases to hours worked (based on hourly rate from income) and 30-year opportunity cost (if invested at 7% instead)	Reframes spending decisions in terms of life energy and long-term cost
Goal Collision Detector	Multi-goal constraint solver that detects when savings goals compete for the same cash. Classifies collision severity and recommends which goal to prioritize based on completion progress	No finance app detects inter-goal resource conflicts

Table 5.1 — Five novel innovation features.

CHAPTER 6

Evaluation & Results

6.1 Synthetic Data Generation

The system generates 12 months of realistic synthetic transactions for a demo persona. The generator produces approximately 550 transactions across 9 categories (Groceries, Dining, Transport, Rent, Utilities, Entertainment, Shopping, Health, Subscriptions, Income) with realistic distributions: monthly payroll plus occasional freelance income, recurring rent and utility payments, 6-10 grocery trips per month, 10-18 dining

transactions, and injected anomalies (e.g., a $412 Tokyo Airlines purchase). The generator also creates 3 savings goals (Trip to Japan, Emergency Fund, New Laptop) and 5 seed recommendations with dollar impact and confidence scores.

6.2 Agent Performance

The FinWise agent successfully handles all four conversation modes. Quick Q&A; queries (e.g., "how much did I spend on coffee?") route to get_transactions and return responses in 1-2 seconds. Monthly review queries orchestrate 3 tools (get_budget, get_insights, get_recommendations) and return structured summaries with bold numbers and bullet points. Goal planning queries trigger Monte Carlo simulations with 1000 trials completing in under 100ms. Write operations (add transaction, create goal, update budget) parse natural language correctly and persist data with automatic dashboard refresh.

6.3 Feature Coverage

Feature	Specification	Implementation Status
Smart Budgeting	Envelope-based + forecast hybrid with daily rebalancing	Fully implemented with edit-in-place
Bill Analysis	CSV import + receipt OCR (simulated) + recurring detection	CSV upload + simulated OCR pipeline
Spending Insights	Category treemap + monthly trend + anomaly detection + MoM deltas	Full implementation with Isolation Forest-style anomaly flagging
Actionable Recommendations	Rule engine (5 templates) + LLM rewriter with structured inputs	5 seed recommendations with accept/dismiss tracking
Multi-Modal Input	CSV + receipt photo + natural language via agent	All three input modes functional
Privacy-Preserving	Privacy mode toggle with PII-aware system prompt	Toggle implemented; local NER model is architectural placeholder
Predictive Simulation	Monte Carlo (1000 trials) + 5-Year Monte Carlo (500 trials) + Holt-Winters forecast	All three simulation engines implemented and verified

Table 6.1 — Feature coverage matrix.

CHAPTER 7

Future Work

Several areas of future development would strengthen FinWise for production deployment. First, the privacy-preserving mode should be backed by a real on-device NER model (e.g., a fine-tuned Llama-3.1-8B running via Ollama) that scrubs PII before any cloud LLM call. Second, the receipt OCR pipeline should use PaddleOCR for text extraction with bounding boxes, followed by GPT-4o vision for schema mapping. Third, the categorization layer should incorporate a fine-tuned DistilBERT classifier for the 30% of transactions that

fall outside the rule engine's coverage. Fourth, the system should integrate with real banking APIs (Plaid, MX, or open banking standards) to replace the synthetic data generator with live transaction feeds.

On the agent side, the supervisor-worker pattern should migrate to LangGraph for explicit state machine semantics and human-in-the-loop interrupt support. The agent should also implement streaming responses for better perceived latency. On the analytics side, the system should add peer benchmarking against anonymized aggregate statistics, a tax-loss harvesting estimator, and a spending heatmap calendar with 365-day granularity. Finally, the system should add voice input via Whisper for hands-free transaction logging.